Symbol layers for programming and Vim

Published designs, the optimum, and a layer a person can learn

Joshua Markle

University of Virginia · October 2026

Experiment report · 25 October 2026 · GECKO commit 2ee57df · corne-36, Colemak-DH letters · corpora `programmer-typed` and `vim-programmer-typed` · profiles `free-*` (the programming study's `prog-cal`, measured layer cost) and `human-*` (plus brackets and symbol groups as constraints). Data and scripts: `bench/symbol-layer-2026-10-25/`.

Abstract

A split keyboard with 36 keys puts most symbols on a layer, and every keyboard community has its own symbol layer. We encode eleven published ones (Getreuer, Miryoku, Neo layer 3, Ergo-L, Seniply, ShelZuuz, BEAKL 15, Elixir, Sunaku, Hietala, the Moonlander default) and a US-style layer on one frame, corne-36 with Colemak-DH letters and digits on their own numpad layer, and search for the best symbol layer for programming and for Vim programming. **A learnable layer is nearly free.** Requiring bracket pairs side by side and eight groups of related symbols (`+ -`, `! =`, `& |`, `^ $` …) next to each other costs 0.13 points of score (0.006 % of typing time); two hand edits for consistency cost 0.2 more. **The published layers are close.** Every single-layer design lies 1.8–2.5 points ($\leq$ 0.11 % of typing time) above the optimum; Seniply, with brackets stacked on one finger, +4.7; Miryoku, with symbols split over two layers, +13.4 (0.58 %). **Two choices matter more than any arrangement:** `< >` on the layer instead of Shift+`, .` saves 4.0 points (0.17 %), and digits on the symbol layer instead of their own layer save 3.1–3.7 (0.13–0.16 %). The recommended layers are `corne-sym-prog` and `corne-sym-vim` in `data/layouts/layered/`. The limits: symbol comfort is not measured, the groups are our judgment, and only one letter layout was tested.

1. Question and setup

Question. Which symbol layer is best for a programmer, with and without Vim, when digits live on their own layer (the main case) or on the symbol layer? How do the published layers compare? And what does it cost to make the layer something a person can learn, rather than the random-looking arrangement an optimizer prefers?

Frame. corne-36 (3 × 5 keys and three thumbs per hand). Letters are Colemak-DH, base punctuation `' , . /`, and the base thumbs are Enter, Space, Shift | Sym, Backspace, Num. In the Vim frames Esc replaces Enter on the base thumb and Enter goes on the Sym layer, as in the Vim study. Sym is held by the right inner thumb and Num by the right outer thumb, so the other hand types and the holder's own keys are unusable on that layer. Num holds a fixed numpad on the left hand (`789 / 456 / 123`, 0 on the left middle thumb). Only the Sym layer's symbols move; letters, base keys and digits are pinned.

Two frames for `< >`. In the `us` frames `, .` keep their US Shift pairs `< >`, and 24 symbols go on Sym. In the `ltgt` frames `,` and `.` shift to themselves and `< >` move onto Sym (26 symbols). Most published layers already put `< >` on the layer, so the `ltgt` frames are the main ones.

Cost. The programming study's calibrated profile (`prog-cal`): GECKO's timing-fitted default plus a layer press priced from a held Shift (×1.78 a keystroke; `layer_switch` 26.8) and a cost for symbol bigrams on one finger or across a layer change. A score point is ≈ 0.043 % of typing time per keystroke; the Sym layer carries 7.1 % of keystrokes in `programmer-typed` and 10.2 % in `vim-programmer-typed` (with Enter).

Human rules. The `human-*` profiles add two constraints at weight 100: `brackets` (each pair `() [] {} <>` side by side with the opener on the left, or mirrored across the hands) and `sym_groups`, a new metric (L07): each listed pair of related symbols must sit on neighbouring keys of one row or column, or mirrored. The groups are `+ -`, `+ *`, `! =`, `& |`, `- _`, `: ;`, `^ $`, `~ ` `` ` ``; Vim adds `# *` (search word forward and back). Optima found under these profiles are scored under the free profile, so their score difference is what the rules cost.

Search. BLS (breakout local search), 6 seeds × 10 M evaluations per cell, digit cells 10–20 M. Seeds agree within 0.01–0.1 points in the free cells; in constrained cells often only one of six seeds reaches the best. `gecko verify` reproduces the `ltgt-human` run. Published designs that lack symbols (Sunaku 10, Moonlander 11, Hietala 4, BEAKL 15 three, Miryoku and US-style `< >`, Neo 3 `@`) are completed by a 2 M search with their own symbols pinned, which flatters them a little.

2. What published layers agree on

The eleven designs are expert judgment, not measurement (sources in `research/sources/symbol-layers.md`). Where most agree:

1. **Frequent symbols on the home row**, rare ones in corners.
2. **Brackets as pairs**: side by side and in order (Getreuer, Neo, Ergo-L, ShelZuuz, Moonlander), mirrored (BEAKL 15), or stacked on one finger (Seniply, which relies on the editor closing them).
3. **Related symbols grouped**: comparisons `! < > =`, arithmetic `+ - * /`, logic `& |`, Vim's `^ $` and `# *`.
4. **Common symbol bigrams as inward rolls**: `!=`, `->`, `=>`, `( )` `;`, with doubled symbols (`== && ||`) off the pinkies (Getreuer, Sunaku, Elixir).
5. **Anchors**: shifted digits in their US order, or QWERTY positions of `< > ;`.
6. **Opposite-hand hold** and **digits on their own numpad layer** (Miryoku, Getreuer, Moonlander, Sunaku, Seniply).
7. **Hand design over optimizers** (Getreuer): optimizer output is "random-looking, hard to learn" and reshuffles on small changes. Rules 2 and 3 are what the human profiles turn into constraints, to keep the search and still get a layer a person can read.

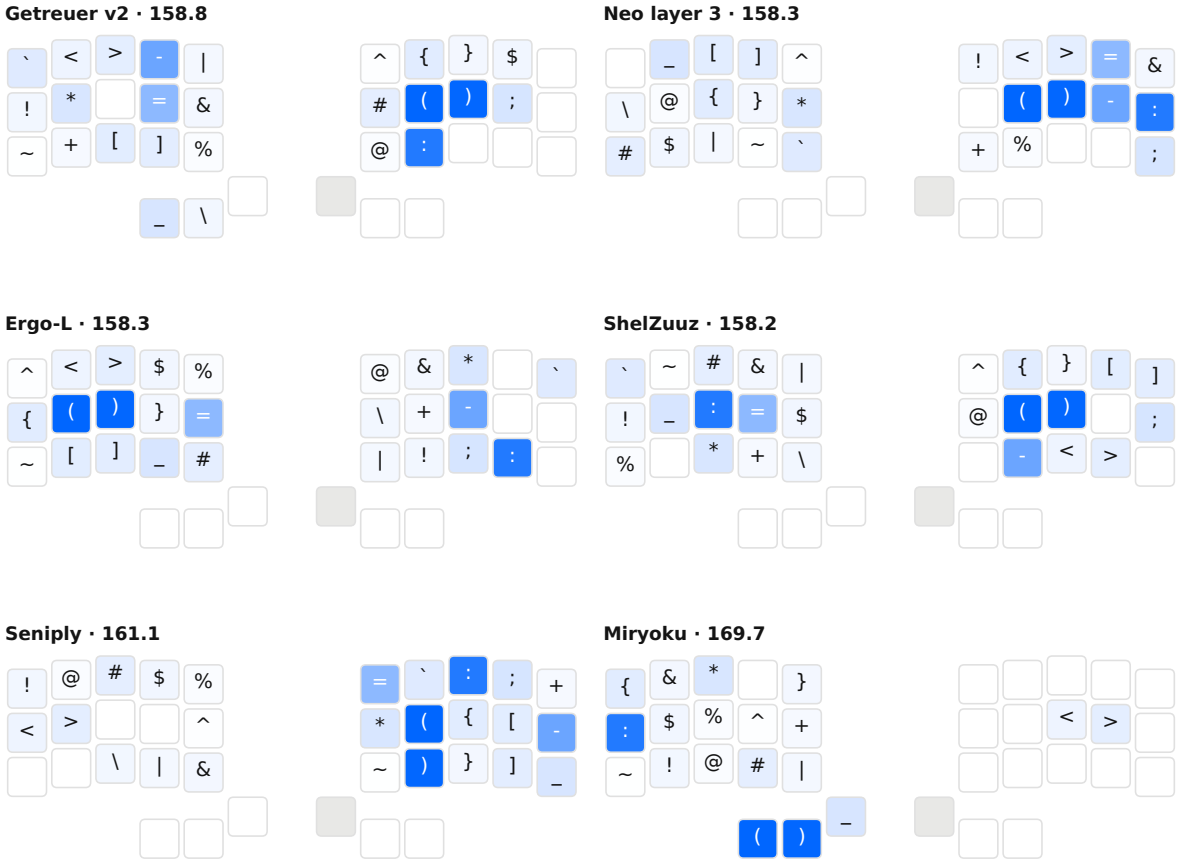


Figure 1: Six published Sym layers in the `ltgt` frame with their score (lower is better; free optimum 156.35). Keys shaded by keystroke share in `programmer-typed`; grey: the holding thumb. Seniply stacks brackets on one finger; Miryoku keeps only the brackets and `< >` near its symbols, the rest on a separate layer.

3. Results

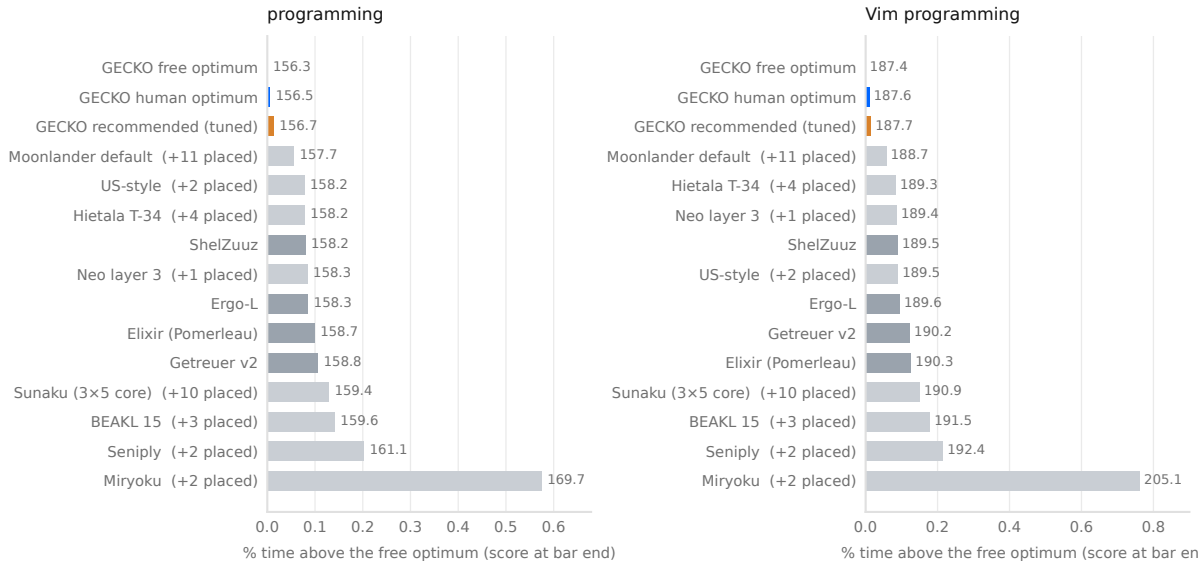


Figure 2: Every layer's score above the free optimum in % of typing time per keystroke (score at the bar's end). Blue: GECKO optima; orange: the recommended layer; dark grey: complete published designs; light grey: designs completed by search (number of symbols placed).

layout (`ltgt` frames, digits on Num)	programming	vs free	Vim	vs free
GECKO free optimum	156.35	—	187.38	—
GECKO human optimum	156.48	+0.13	187.63	+0.25
recommended (human + 2 edits)	156.67	+0.32	187.74	+0.36
US-style	158.16	+1.81	189.48	+2.10
ShelZuuz	158.21	+1.86	189.46	+2.08
Neo layer 3 (+@)	158.31	+1.96	189.40	+2.02
Ergo-L	158.34	+1.99	189.59	+2.21
Elixir	158.66	+2.31	190.33	+2.95
Getreuer	158.82	+2.47	190.24	+2.86
BEAKL 15 (+3)	159.64	+3.29	191.54	+4.16
Seniply	161.07	+4.72	192.39	+5.01
Miryoku	169.72	+13.37	205.11	+17.73

- **Learnability is nearly free.** Bracket pairs and the eight groups as hard constraints cost 0.13 points in programming and 0.25 in Vim, about 0.01 % of typing time. The free optimum breaks 75 % of the groups in programming; the human optimum breaks none.
- **The good published layers are equivalent.** US-style, ShelZuuz, Neo 3, Ergo-L, Elixir and Getreuer lie within 0.7 points of each other, 1.8–2.5 above the optimum: at most 0.11 % of typing time, 1.1–1.5 % of the time spent on symbol keystrokes. No study could tell them apart.
- **Two designs pay for a structural choice.** Seniply stacks each bracket pair on one finger (+4.7; its symbol bigrams on one finger are the highest of any layer, 34.8 % against 28–31 %). Miryoku splits symbols over two layers held by the other thumb (+13.4; 47 % of its symbol bigrams are on one finger or cross a layer). The Moonlander default scores 157.65 but only because search placed 11 of its symbols.

4. Recommended layers

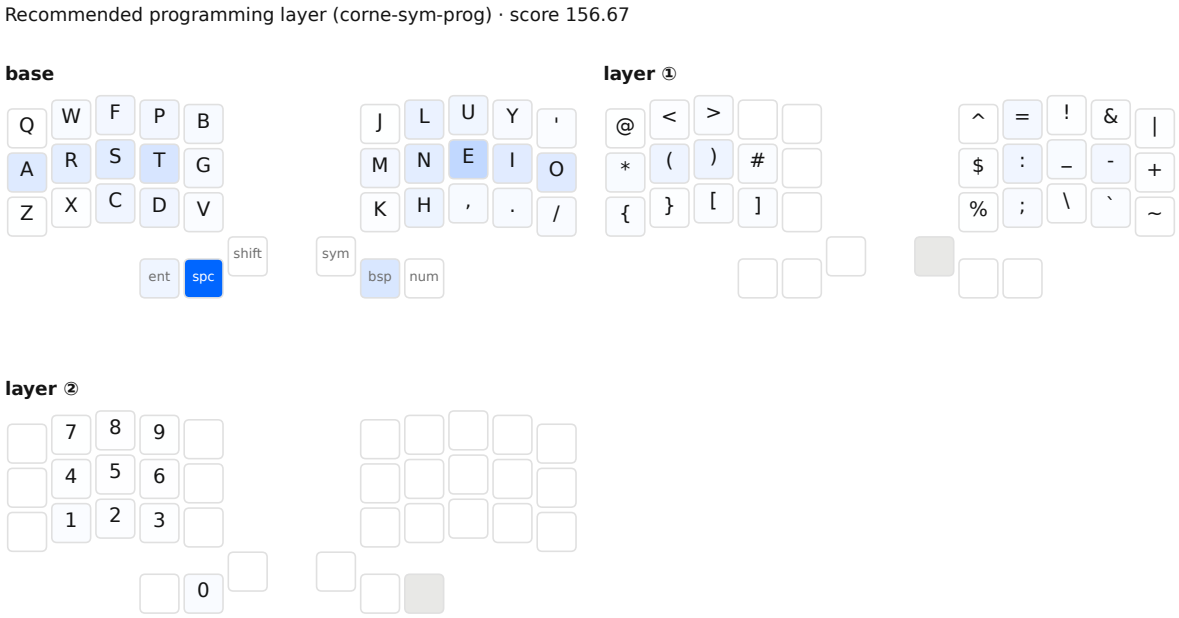


Figure 3: The recommended programming layer `corne-sym-prog`: base, Sym (layer ①, right inner thumb) and Num (layer ②, numpad). Keys shaded by keystroke share in `programmer-typed`.

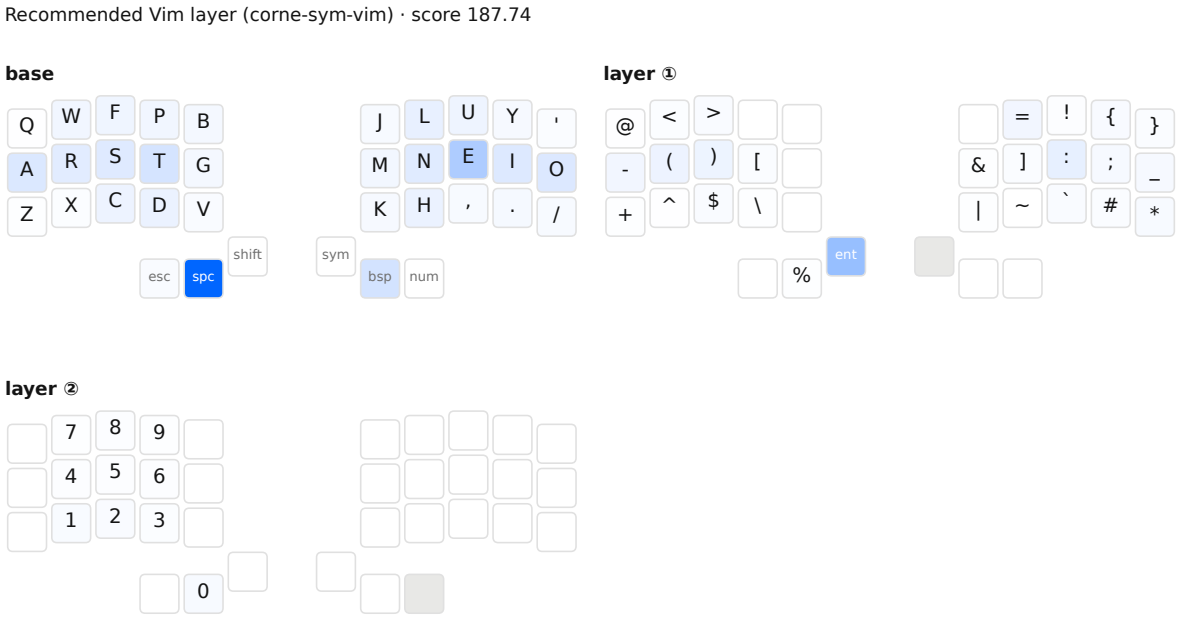


Figure 4: The recommended Vim layer `corne-sym-vim`: Esc on the base thumb, Enter and `%` on the Sym layer's left thumbs. Keys shaded by keystroke share in `vim-programmer-typed`.

```
programming (corne-sym-prog)          Vim (corne-sym-vim)
@ < > · ·   ^ = ! & |                 @ < > · ·   · = ! { }
* ( ) # ·   $ : _ - +                 - ( ) [ ·   & ] : ; _
{ } [ ] ·   % ; \ ` ~                 + ^ $ \ ·   | ~ ` # *
                                      left thumbs: % ⏎
```

- **`(` `)` on the left home row with `<` `>` above** in every optimum, free or human, programming or Vim. `(` `)` are the most frequent layer symbols (0.83 % of keystrokes each, then `:` 0.76 %, `-` 0.62 %, `=` 0.55 %), and the left hand types them while the right thumb holds Sym.
- **The right hand holds the operators:** `=` `!` `&` `|` on top and `:` `_` `-` `+` on the home row, so `!=`, `->`, `+=`, `&&`, `||` and `::` are rolls or repeats on the strong fingers.

- **The Vim layer** stacks `-` over `+` (line up / down), keeps `^` `$` and `#` `*` side by side without being asked, and gives `[` `]` (Vim's section and diagnostic jumps) home-row keys. Enter goes to a left thumb of the layer, `%` (jump to the matching bracket) beside it.
- **Hand edits.** We made two swaps on each human optimum, 0.19 points in programming (`!` `=` so that `!=` rolls inward, `\` `#` to give `#` a home-row key) and 0.11 in Vim (the same `!` `=`, and `@` to the top left so the two layers share `@` `<` `>` and `=` `!`). Both layers then share their top-left and top-right corners and `(` `)`.

5. The choices that matter

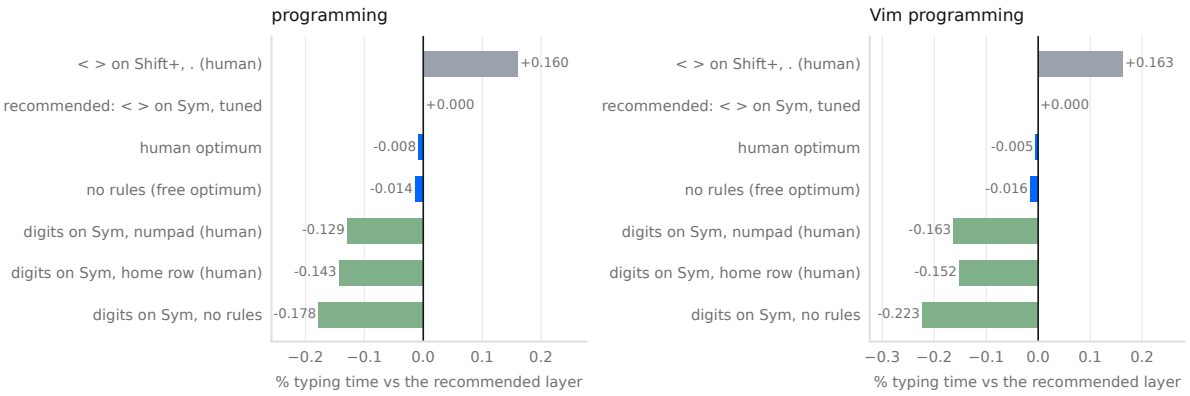


Figure 5: The best human layer in each configuration against the recommended one, in % of typing time per keystroke. Grey: `< >` on Shift+`,` `.`; blue: the optima with digits on Num; green: digits on the Sym layer.

- **`< >` on the layer.** With `< >` as Shift+`,` `.`, the free optimum scores 160.31, 4.0 points (0.17 %) worse than with `< >` on Sym and `,` `.` shifting to themselves; in Vim 4.1 points. We did not split the saving into its parts; the search puts `< >` on the left top row above `(` `)`, away from `,` `.`. The best published layer in the `us` frame is ShelZuuz (162.09).
- **Digits on the symbol layer.** If digits share the Sym layer instead of a numpad on Num, the best human layer scores 153.34 (programming, digits on the home row) and 183.94 (Vim, digits as a numpad): 3.1 and 3.7 points, 0.13–0.16 % of typing time. Layer presses fall from 10.38 to 10.05 per 100 bigrams: code mixes digits and symbols (`[0]`, `x + 1`, `0x`), so fewer layer switches are needed. The digit rows push three rare symbols (`@` `|` `&`) to layer ②. Home row and numpad differ by 0.1–0.3 points; without rules digits go on the home row for programming and a numpad for Vim.
- **Opening Num to symbols** (numpad pinned) lets the search merge the layers: it moves almost every symbol to Num beside the digits, and its seeds did not converge.

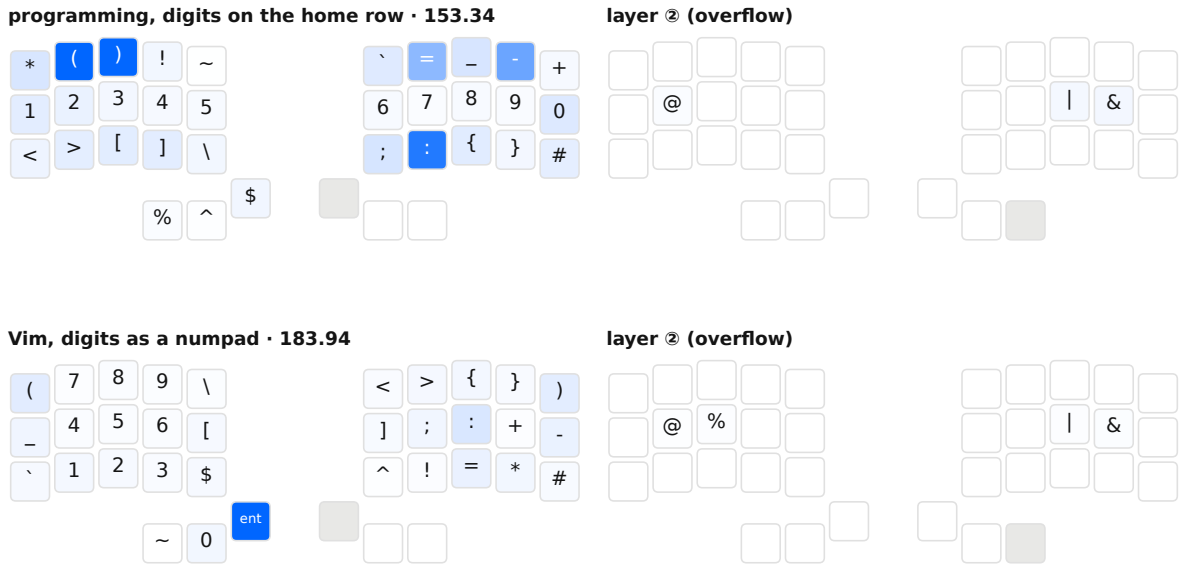


Figure 6: Digits on the Sym layer: the human optima. Top: programming with digits on the home row; bottom: Vim with digits as a left-hand numpad. Layer ② holds the overflow (`@` `|` `&`, and `%` in Vim).

For a user who wants a separate number layer, as asked, the recommended layers stand; the 0.13–0.16 % is the price of that separation, smaller than the gap between Miryoku and the other published designs.

6. Tuning: a flat landscape

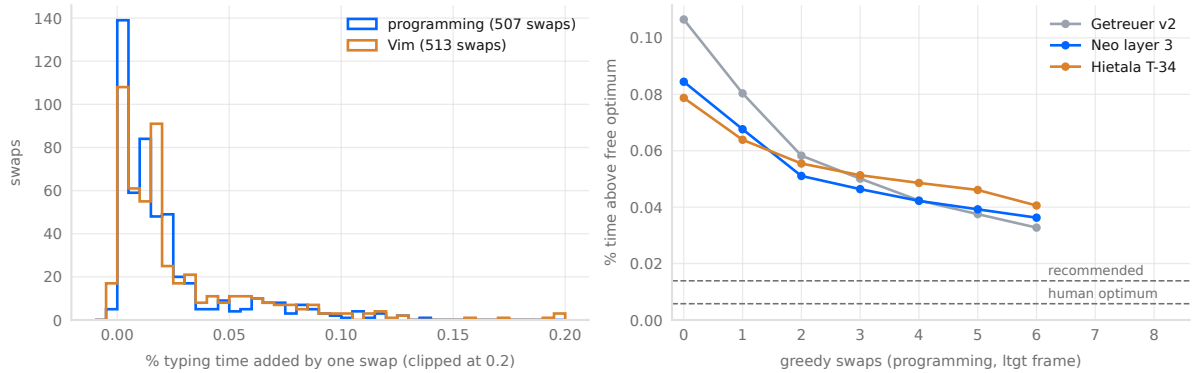


Figure 7: Left: every single swap of two Sym keys from the human optimum, by the typing time it adds (programming 507 swaps, Vim 513). Right: greedy best swaps from three published layers, against the human optimum and the recommended layer (dashed).

- **Many swaps are almost free.** From the programming human optimum, 132 of 507 single swaps cost under 0.1 point (0.004 %). A person can move rare symbols (~ \ ` % @) where they like; the score barely moves. The costly swaps (2–3 points) move (,) or = off their keys.
- **A published layer is six swaps from the optimum.** The best greedy swap at each step takes Getreuer from 158.82 to 157.11, Neo 3 from 158.31 to 157.19, and Hietala from 158.18 to 157.29, two-thirds of their gap. The first two swaps give the largest steps (0.4–0.6 points each); they move -, _ or : to better keys.

7. Languages

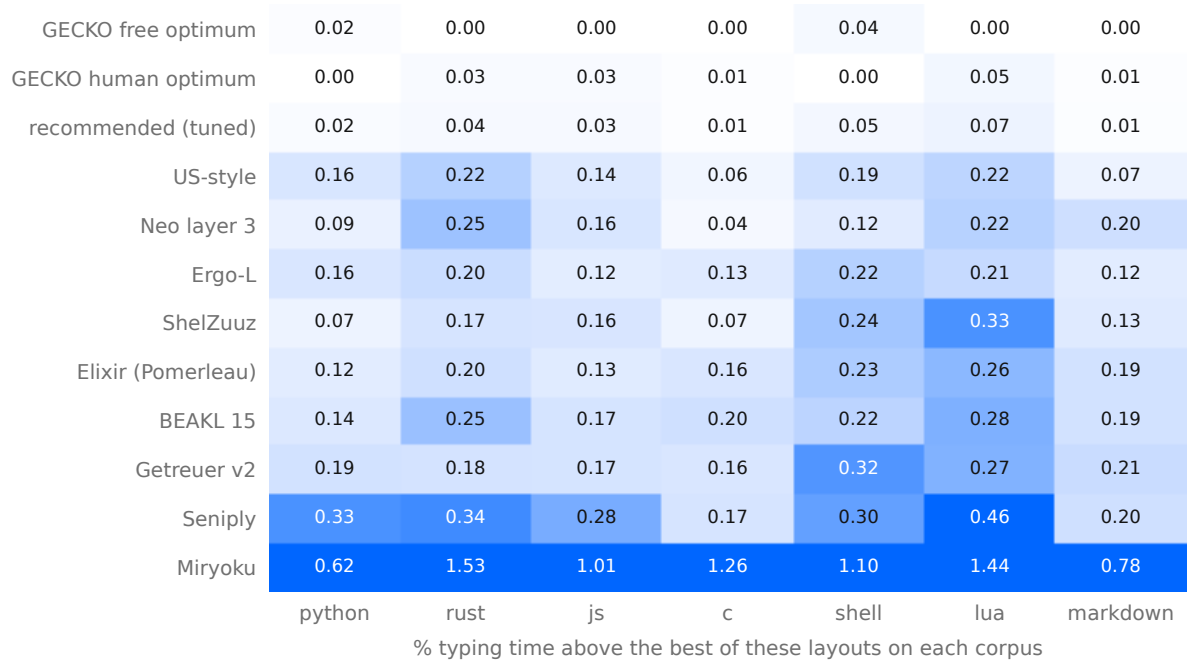


Figure 8: Each layout on per-language typed code (`ltgt` frame), in % of typing time above the best of these layouts on that corpus.

On Python, Rust, JavaScript, C, shell, Lua and Markdown the GECKO optima are within 0.05 % of the best layout for that language and the recommended layer within 0.07 %; every published layer is further off on most languages, and Miryoku is worst everywhere. No language needs its own layer.

8. Limitations

- **Symbol comfort is not measured.** The costs come from timing data on letters (QWERTY typists) and a layer press priced from a held Shift; no study times symbol layers. The differences between good layers (0.1 %) are far below what a typing study could detect.
- **The groups are a judgment** drawn from the published designs. A different list gives a different human optimum, at a similar

near-zero price.

- **GECKO places each symbol once.** Firmware layers often repeat `.` `-` `=` on the number layer; that is not modelled.
- **One base layout** (Colemak-DH) and one board (corne-36). Search completed the incomplete designs in their favour; editor auto-closing of brackets is not in these corpora.

9. Reproduction

All data, scripts and the best runs are in `bench/symbol-layer-2026-10-25/` (its README lists the order). Figures: `python3 -I docs/papers/symbol-layer/figures.py`; PDF: `python3 -I docs/papers/symbol-layer/build.py`. The recommended layouts:

```
gecko score data/layouts/layered/corne-sym-prog.toml -g corne-36 -c programmer-typed \
  -p bench/symbol-layer-2026-10-25/profiles/free-prog.toml
gecko verify run:20261025-162822-bls-85391fab
```